FastAPI E-Commerce Backend - Task Description

Project Overview

Build a RESTful E-Commerce Web API using FastAPI with JWT authentication, similar to the .NET Core backend you built. The API should support product management, category management, user authentication, shopping cart, and order processing.

Requirements

1. Authentication & Users

Endpoints:

- POST /api/auth/sign-up - User registration
- POST /api/auth/login - User login

Technical Requirements:

- Store passwords securely using bcrypt hashing
- Return JWT token upon successful login
- Protect secured endpoints using JWT authentication
- Token should expire after configured time

Example Request/Response:

Sign-Up:

json

```
// Request
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "SecurePass123!"
}

// Response
{
  "id": 1,
```

```
  "name": "John Doe",
  "email": "john@example.com"
}
```

Login:

json

```
// Request
{
  "email": "john@example.com",
  "password": "SecurePass123!"
}

// Response
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 1440,
  "email": "john@example.com"
}
```

---

2. Products Module

Endpoints:

- GET /api/product - Get all products (paginated)
- GET /api/product/{id} - Get product by ID
- POST /api/product - Create product (Admin only)
- PUT /api/product/{id} - Update product (Admin only)
- DELETE /api/product/{id} - Delete product (Admin only)

Technical Requirements:

- Pagination: `page` and `size` query parameters
- Default: `page=1`, `size=10`
- Max `size=100`
- Include category name in response
- Validate all inputs

Example Response:

```json
{
  "items": [
    {
      "id": 1,
      "name": "iPhone 15",
      "price": 999.99,
      "quantity": 50,
      "category_id": 1,
      "category_name": "Electronics"
    }
  ],
  "total_count": 100,
  "page_number": 1,
  "page_size": 10,
  "total_pages": 10,
  "has_previous_page": false,
  "has_next_page": true
}
```

---

3. Categories Module

Endpoints:

- GET /api/category - Get all categories
- GET /api/category/{id} - Get category by ID
- POST /api/category - Create category (Admin only)
- PUT /api/category - Update category (Admin only)
- DELETE /api/category/{id} - Delete category (Admin only)

Validation Rules:

- Category name must be unique
- Category name cannot be empty

---

4. Cart Module

Endpoints:

- GET /api/cart/{user_id} - Get current user's cart
- POST /api/cart/add?userID={id} - Add product to cart
- PUT /api/cart/quantity - Update product quantity
- POST /api/cart/checkout - Checkout cart

Business Rules:

- Cannot add more than available stock
- Cannot checkout empty cart
- Cart items with product details (name, price, subtotal)
- Checkout reduces product stock

Example Cart Response:

json

```
{
  "id": 1,
  "user_id": 1,
  "total_price": 149.98,
  "items": [
    {
      "id": 1,
      "product_id": 1,
      "product_name": "iPhone 15",
      "product_price": 999.99,
      "quantity": 1,
      "subtotal": 999.99
    }
  ]
}
```

---

5. Order Module

Endpoints:

- GET /api/order - Get all orders for current user
- GET /api/order/{id} - Get order by ID

Order Status:

- Pending
- Processing
- Shipped
- Delivered
- Cancelled

Example Order Response:

json

```
{
  "id": 1,
  "user_id": 1,
  "order_date": "2024-01-15T10:30:00",
  "status": "Pending",
  "price": 1049.98,
  "items": [
    {
      "product_id": 1,
      "product_name": "iPhone 15",
      "quantity": 1,
      "unit_price": 999.99,
      "subtotal": 999.99
    }
  ],
  "shipping_address": "123 Main St",
  "payment_method": "Credit Card"
}
```

Technical Requirements

1. Framework & Libraries

- FastAPI - Web framework
- SQLAlchemy - ORM (async/sync)

- Alembic - Database migrations
- Pydantic - Data validation
- python-jose - JWT handling
- passlib[bcrypt] - Password hashing

2. Database

- PostgreSQL or SQLite (for development)
- Define all models with relationships
- Use indexes on foreign keys and frequently searched columns

3. Database Models

Required Tables:

- `users` (id, name, email, password_hash, created_at)
- `categories` (id, name)
- `products` (id, name, price, description, quantity, category_id, created_at)
- `carts` (id, user_id, created_at)
- `cart_items` (id, cart_id, product_id, quantity)
- `orders` (id, user_id, order_date, status, price, shipping_address, payment_method)
- `order_items` (id, order_id, product_id, quantity, unit_price)

Relationships:

- User (1) → Cart (1)
- User (1) → Orders (∞)
- Category (1) → Products (∞)
- Cart (1) → CartItems (∞) → Product (1)
- Order (1) → OrderItems (∞) → Product (1)

Security Requirements

1. JWT Configuration

- Secret key in environment variables
- Algorithm: HS256
- Expiration: Configurable (default 24 hours)

- Store token in Authorization header: `Bearer {token}`

2. Password Security

- Hash passwords using bcrypt
- Never return password in responses

3. Input Validation

- Validate all request bodies using Pydantic
- Email validation
- Password length validation (min 6 characters)
- Price > 0 validation
- Quantity >= 0 validation

4. CORS

- Configure CORS for frontend domains
- Allow credentials

🚦 Error Handling

Global Exception Handler

- Catch all exceptions and return consistent format
- Map custom exceptions to HTTP status codes:

Exception	Status Code
NotFound	404
BadRequest	400
Conflict	409
Unauthorized	401
Validation Error	400

Internal Server Error	500

Error Response Format:

json

```
{
  "message": "Product with ID 999 not found",
  "status_code": 404,
  "error_type": "NotFoundException",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

DTOs / Schemas (Pydantic Models)

Required Schemas:

Auth:

- `LoginRequest` (email, password)
- `RegisterRequest` (name, email, password)
- `LoginResponse` (access_token, token_type, expires_in, email)
- `UserResponse` (id, name, email)

Product:

- `ProductResponse` (id, name, price, quantity, category_id, category_name)
- `CreateProductRequest` (name, price, description, quantity, category_id)
- `UpdateProductRequest` (name, price, description, quantity, category_id)

Category:

- `CategoryResponse` (id, name)
- `CreateCategoryRequest` (name)

Cart:

- `CartResponse` (id, user_id, total_price, items)
- `CartItemResponse` (id, product_id, product_name, product_price, quantity, subtotal)
- `AddToCartRequest` (product_id, quantity)
- `UpdateQuantityRequest` (cart_item_id, quantity_required)

Order:

- `OrderResponse` (id, user_id, order_date, status, price, items)
- `OrderItemResponse` (product_id, product_name, quantity, unit_price, subtotal)
- `CheckoutRequest` (user_id, payment_method, shipping_address)

Pagination:

- `PaginatedResponse` (items, total_count, page_number, page_size, total_pages, has_previous_page, has_next_page)